

PLAGIARISM DETECTION SYSTEM

Assignment Report

1. Introduction

Plagiarism is the act of copying someone else's work, ideas, or expressions without proper acknowledgment. With the rapid growth of digital content, plagiarism has become a serious issue in academics, research, and professional environments. Detecting plagiarism manually is time-consuming and inefficient, especially when dealing with large datasets or documents.

A **Plagiarism Detection System** is a software solution designed to automatically identify similarities between documents and detect potential cases of copied content. This system compares submitted text with existing documents, databases, or web sources and provides a similarity score along with highlighted matched content.

The goal of this project is to build an efficient and scalable plagiarism detection system that can analyze text documents (PDF/Word), compute similarity scores, and present results in a user-friendly manner.

2. Objectives

The main objectives of the Plagiarism Detection System are:

- To detect similarity between multiple documents.
 - To identify copied or duplicated content.
 - To support multiple file formats such as PDF and Word.
 - To provide a similarity percentage score.
 - To highlight plagiarized sections.
 - To ensure fast and accurate comparison.
 - To build a scalable and deployable system.
-

3. Technologies Used

3.1 Frontend

- **HTML, CSS, JavaScript**
 - Used to design the user interface.
 - Provides input forms for uploading documents.
 - Displays results in a clean and structured format.

3.2 Backend

- **Python**
 - Core programming language used for processing.
 - Handles file reading, text extraction, and similarity computation.
- **Framework (Optional: Flask / FastAPI)**
 - Used to create APIs for communication between frontend and backend.
 - Handles file uploads and responses.

3.3 Text Processing

- **NLTK (Natural Language Toolkit)**
 - Used for text preprocessing.
 - Tokenization, stopword removal, and normalization.
- **spaCy (Optional)**
 - Advanced NLP processing for better accuracy.

3.4 Similarity Algorithms

- **TF-IDF (Term Frequency - Inverse Document Frequency)**
 - Converts text into numerical vectors.
 - Measures importance of words in documents.
- **Cosine Similarity**
 - Measures similarity between two text vectors.
 - Produces similarity score between 0 and 1.

3.5 File Handling

- **PyPDF2 / pdfplumber**
 - Extracts text from PDF files.
- **python-docx**
 - Extracts text from Word documents.

3.6 Database (Optional)

- **MySQL / SQLite**
 - Stores uploaded documents and comparison results.

3.7 Deployment

- **Cloud Platforms (e.g., Render, AWS, or Vercel)**
 - Used to deploy the application for public access.
-

4. System Architecture

The system follows a **client-server architecture** with the following components:

4.1 User Interface Layer

- Users upload documents through a web interface.
- Input is sent to the backend server.

4.2 Application Layer

- Handles:
 - File uploads
 - Text extraction
 - Preprocessing
 - Similarity computation

4.3 Processing Layer

- Performs:
 - Tokenization
 - Stopword removal
 - Vectorization (TF-IDF)
 - Similarity calculation (Cosine Similarity)

4.4 Data Layer

- Stores:
 - Uploaded documents
 - Extracted text
 - Similarity scores

4.5 Output Layer

- Displays:
 - Similarity percentage
 - Matched content
 - Highlighted plagiarism sections
-

5. Working of the System

The working process of the system can be explained step-by-step:

Step 1: Document Upload

- User uploads one or multiple files (PDF/Word).

Step 2: Text Extraction

- Text is extracted using libraries like PyPDF2 or python-docx.

Step 3: Text Preprocessing

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization

Step 4: Vectorization

- Text is converted into numerical format using TF-IDF.

Step 5: Similarity Calculation

- Cosine similarity is calculated between documents.

Step 6: Result Generation

- Similarity score is displayed.
 - Matching sentences or paragraphs are highlighted.
-

6. Algorithms Used

6.1 TF-IDF Algorithm

TF-IDF is used to evaluate how important a word is in a document relative to a collection of documents.

- **Term Frequency (TF):** Frequency of a word in a document.
- **Inverse Document Frequency (IDF):** Importance of a word across documents.

Formula:

$$TF - IDF = TF \times IDF$$

6.2 Cosine Similarity

Cosine similarity measures the angle between two vectors.

Formula:

$$\text{Similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

- Value ranges from 0 (no similarity) to 1 (identical).
-

7. Features of the System

- Supports multiple file formats (PDF, DOCX)
 - Fast similarity detection
 - Accurate results using NLP techniques
 - User-friendly interface
 - Scalable architecture
 - Real-time analysis
 - Highlights plagiarized content
-

8. Advantages

- Saves time compared to manual checking
 - Improves academic integrity
 - Easy to use
 - Scalable for large datasets
 - Automated and efficient
-

9. Limitations

- Cannot detect paraphrased content perfectly
 - Depends on dataset quality
 - May produce false positives
 - Requires computational resources for large data
-

10. Applications

- Academic institutions for assignment checking
 - Research paper validation
 - Content writing platforms
 - Recruitment and technical assessments
 - Publishing industries
-

11. Future Enhancements

- Integration with web sources for broader detection
 - Use of deep learning models for better accuracy
 - Detection of paraphrased content
 - Real-time API-based plagiarism checking
 - Multi-language support
-

12. Conclusion

The Plagiarism Detection System is an essential tool in today's digital world where content duplication is common. By leveraging Natural Language Processing and machine learning techniques like TF-IDF and cosine similarity, the system efficiently identifies similarities between documents.

This project demonstrates how AI and text processing can be used to solve real-world problems. With further improvements and integration of advanced models, the system can become even more powerful and accurate.

13. References

1. Research papers on TF-IDF and Cosine Similarity
2. Python Documentation
3. NLTK and spaCy Official Docs
4. Machine Learning and NLP textbooks